

Classes

revision: structs are a group of data types bundled in one place

```
ex:
struct student{
string name;
int marks[10];
int ID;};
```

this is useful for Plain old data structures
where it's JUST a bundle of values in one place

but sometimes, programs need, for example, functions related to the structure we're dealing with, and it doesn't make a lot of sense to put the function independently from the structure.

for example: if I want to increment the mark for a student:

```
ex:
struct student{
string name;
int marks[10];
int ID;};

void incMark(student &stud,int index){
    mark = stud->marks[index];
    case stud->marks[index]
    'B': mark = 'a'
    // and so on.... }
```

```
int main(){
student SajElm;
SajElm.marks[2] = 'F';
incMark(SajElm, 2);
```

notice how the struct is SEPERATE from the function
it would be easier to tell him:
"look at SajElm and increment his mark"
than
"increment a mark, let it be sajElm"

ANYWAYS, structs, while good at bundling data, lack some features we will see in classes, including some principles:

CLASSES are also a group of data types in one place, written like this:

ex:

```
class horse{  
    private:  
        muscleMap Muscles;
```

Private members are NOT accessible
outside class
horsey.Muscles will give an error

```
    public:  
        string name;  
        float height;  
        float speed;  
        color furCol;
```

This is the struct we're used to

```
horse(string n, float h  
        float s, color c){  
    name = n;  
    height = h; // & so on  
    Cout << "horse " << this->name << "is here!";  
}
```

this is a constructor, and is called
when the INSTANCE of a class
is CREATED
horse horsey = horse("batee5",220,...)

```
void neigh(){ cout << name << " says: NEEEEEEIIGGGHHH!!!";}
```

this is a function that belongs to a class

```
int main(){
```

```
    horse hosan = horse("horsey",203, 18, BROWN);  
    //couts " Horsey is here!"
```

```
    hosan.neigh()  
    // couts " horsey says: NEEEEEEEEEEEEEEEEIIIIIIIGH " }
```

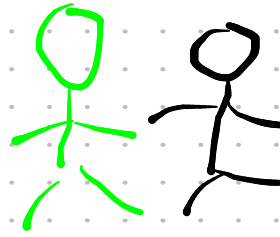
Here are four main features of classes
that differentiate it from structs:

1. Encapsulation

classes act like a black box: you can't see what's inside and you shouldn't.

private class members are as such; doing `classObj.member` is invalid BY DEFAULT unless you declare it as public variable.

programmer



data

interface function

this is good practice of course, as some data types are not supposed to change instantaneously like that; they need an interface function to deal with

ex:

```
class horse{  
    private:  
    muscleMap Muscles;  
    int age;  
    int strength;  
    bool dead;
```

```
    public:  
    string name;  
    float height;  
    float speed;  
    color furCol;
```

YOU can't directly change age or strength or status

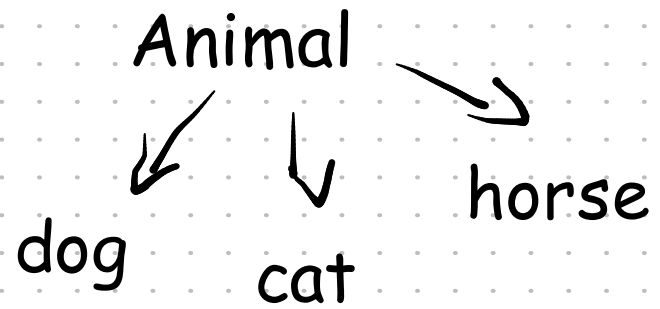
```
horse(string n, float h  
        float s, color c){  
    name = n;  
    height = h; // & so on  
}
```

```
void Older(int timeAge){  
    this->age += timeAge;  
    this->strength -= timeAge;  
    if (this->strength <= 0) this->dead = true; }
```

however, this public function can, and when it does it also decrements the strength automatically, without manually doing this in main

2- inheritance

some classes are very similar to others, so it makes sense that they can inherit some method functions and variables from it



```
ex:
class animal{
public:
int health;
float weight;

void noise(){
cout << "placeholder noise"; }
animal(){ health = 100; weight = 90; } }

class cat : public animal{
void noise(){ cout << "meow"; } }

class dog : public animal{
void noise(){ cout << "woof"; } }

class horse : public animal{
void noise(){ cout << "neigh"; } }
```

for example, ALL animals:

- have a health (/100)
- do a noise
- have a weight

but they're not equal and they produce different noises

notice how we didn't redefine all the methods of animal in it's derived classes?

so now when we do:

```
horse.noise() // outputs neigh
cat.noise() // outputs meow
```

3- Polymorphism

you can redefine functions with the same name but making different procedures if you change how the parameters work

ex:

```
float determinant( matrix2D mat ){  
    return mat.a*mat.d - mat.b*mat.c; }  
  
float determinant ( matrix3D mat ){  
    return mat.a(mat.e*mat.z-mat.f*mat.y) // & so on }  
  
float total( int a, int b ){ return a + b;}  
  
float total ( int a, int b, int c){ return a + b + c;}  
  
int main(){  
    cout << total ( 1,2,3 ) + total ( 4,5 ); }  
}
```

4- abstraction

it's more like an art or a principle but the idea is that your object is like a black box; you only care about the input and output but the internal workings are irrelevant

for example: a vector class has a method to rotate it by 30 deg. what should happen is that the class creates a rotation matrix, and does matrix multiplication to rotate that vector, but you only care about the vector you input, the num of degrees and the output vector

this is useful for making libraries or otherwise code that you may use later, or anyone else. instead of concerning yourself with the internal workings, just think about what you're doing with the result.

Constructors and destructors

these are special functions that get called when an object is created (constructor) or deleted (destructor)

for example: I want to make a matrix class with any size

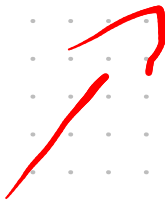
ex:

```
class matrix{
    float** mat;
    int rows;
    int cols;

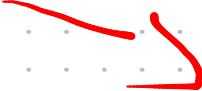
    matrix(int row, int col){
        cout << "mat created";
        rows = row;
        cols = col;
        mat = new float*[rows];
        for ( int i : mat ){
            mat[i] = new float[cols];} }

    ~matrix(){
        for(int i : mat ){
            delete [] mat[i]; }
        delete []mat;
        cout << "mat deleted";}
}
```

automatically makes a dynamic storage for the matrix



automatically DELETES the dynamic storage for the matrix when its scope ends in main()



```
int main(){

    { matrix matt = matrix(3,5);
      // couts "mat created"

    } // couts "mat deleted" when scope ends
```

"This" keyword

"This" is a POINTER that points to the object that has called the function

ex:

```
class matrix{
    float** mat;
    int rows;
    int cols;

// constructor and destructor

    void identity(){
        for(i : mat){ for ( j : mat[i] ) {

            if (i == j){this->mat[i][j] = 1;}
            else{ this->mat[i][j] = 0;};
        }
    }

    int main(){

        { matrix matt = matrix(3,3);
          // couts "mat created"

          matt.identity();

        } // couts "mat deleted" when scope ends
```

* constructors and
destructors do NOT have this.
they can access the members directly

this changes the elements
inside the object itself

its like calling
matrix matt = matrix(3,3);

matt.mat[i][j]

but inside the class matt
is 'this' instead

Header files

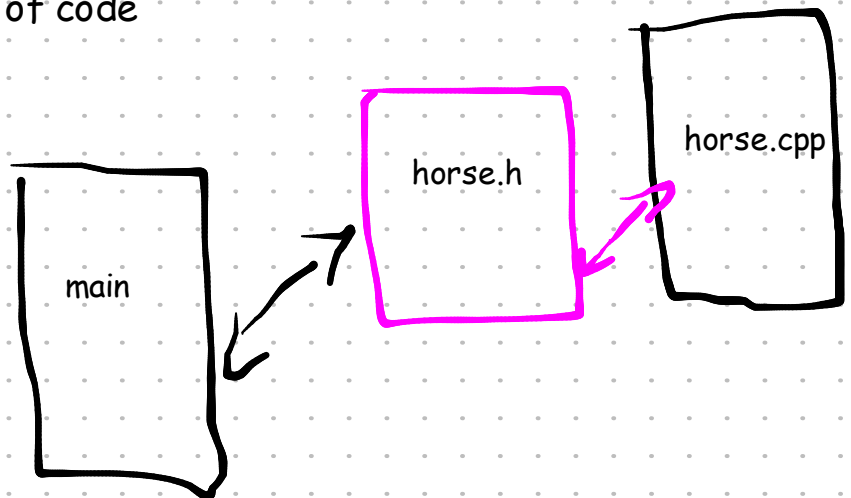
for big projects: it's best to leave everything as separate files rather than one big chunk of code

```
#include "horse.cpp"
```

ex:

```
// horse.h folder
```

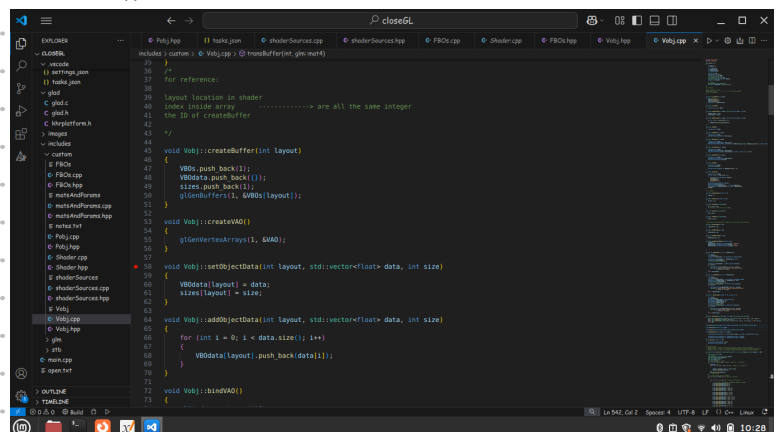
```
class horse{  
int health;  
int age;  
  
void age(int ); }  
  
//notice how its declarations only without details?
```



header files act as an interface that tells the main folder "hey! this is a class that has the following variables and method functions" so it can use them, but the details inside is compiled in the cpp file

ex: my project's .h and .cpp files for a class I made:

C++ file: notice the details inside the functions



.h file:
notice that only the function names are there

